

HDOC 33

Name: Keshav Mittal

SAP ID: 590032934

Batch: 16

1. Algorithm

1. Start main program.
2. Display the menu with options 1 to 3.
3. Read the user's choice and use a switch statement.
4. Case 1 (Addition Associativity):
 - Read dimensions m and n . Declare matrices A , B , C of size $m \times n$.
 - Read elements for matrices A , B , and C .
 - Compute $AB = A + B$, then $LHS = AB + C$.
 - Compute $BC = B + C$, then $RHS = A + BC$.
 - Compare LHS and RHS matrices. If equal, print that matrix addition is associative.
5. Case 2 (Multiplication Associativity):
 - Read dimensions m , n , p , q . Declare matrices $A(m \times n)$, $B(n \times p)$, $C(p \times q)$.
 - Read elements for matrices A , B , and C .
 - Compute $AB = A * B$ (size $m \times p$), then $LHS = AB * C$ (size $m \times q$).
 - Compute $BC = B * C$ (size $n \times q$), then $RHS = A * BC$ (size $m \times q$).
 - Compare LHS and RHS matrices. If equal, print that matrix multiplication is associative.
6. Case 3 (Exit): Terminate the program.
7. Stop

2. Test-Case Table

TC ID	Scenario	Inputs	Expected Output
TC-01	Addition Associativity	Choice: 1. $m=2, n=2$. $A=[1 \ 1; 1 \ 1]$, $B=[2 \ 2; 2 \ 2]$, $C=[3 \ 3; 3 \ 3]$	$(A+B)+C = [6 \ 6; 6 \ 6]$, $A+(B+C) = [6 \ 6; 6 \ 6]$. Addition is associative.
TC-02	Multiplication Associativity	Choice: 2. $m=2, n=2, p=2, q=2$. $A=[1 \ 0; 0 \ 1]$, $B=[2 \ 0; 0 \ 2]$, $C=[3 \ 0; 0 \ 3]$	$(A*B)*C = [6 \ 0; 0 \ 6]$, $A*(B*C) = [6 \ 0; 0 \ 6]$. Multiplication is associative.
TC-03	Exit	Choice: 3	Exiting program...

3. C Program Source Code

```
#include <stdio.h>

void readMatrix(int r, int c, int mat[r][c]); void printMatrix(int r, int c,
int mat[r][c]);
void addMatrices(int r, int c, int m1[r][c], int m2[r][c], int res[r][c]);
void multiplyMatrices(int r1, int c1, int c2, int m1[r1][c1], int m2[c1][c2], int res[r1][c2]);
int compareMatrices(int r, int c, int m1[r][c], int m2[r][c]); void
verifyAdditionAssociativity(void);
void verifyMultiplicationAssociativity(void);

int main() {
    int choice; do {
        printf("\n--- Matrix Associative Properties Menu ---\n");
        printf("1. Verify associative property of matrix addition: (A+B)+C = A+(B+C)\n"); printf("2. Verify associative property of matrix
multiplication: (A*B)*C = A*(B*C)
\n");

        printf("3. Exit\n"); printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) { case 1:
            verifyAdditionAssociativity(); break;
            case 2:
            verifyMultiplicationAssociativity(); break;
            case 3:
            printf("Exiting program...\n"); break;
            default:
            printf("Error: Invalid choice.\n");
        }
    } while (choice != 3);

    return 0;
}

void readMatrix(int r, int c, int mat[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { scanf("%d", &mat[i][j]);
    }
}
}

void printMatrix(int r, int c, int mat[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { printf("%d\t", mat[i][j]);
    }
    printf("\n");
}
```

```

    }
}

void addMatrices(int r, int c, int m1[r][c], int m2[r][c], int res[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { res[i][j] = m1[i][j] + m2[i][j];
    }
}
}

```

```

void multiplyMatrices(int r1, int c1, int c2, int m1[r1][c1], int m2[c1][c2], int res[r1][c2]) {
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) { res[i][j] = 0;
            for (int k = 0; k < c1; k++) { res[i][j] += m1[i][k] *
                m2[k][j];
            }
        }
    }
}
}

```

```

int compareMatrices(int r, int c, int m1[r][c], int m2[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { if (m1[i][j] !=
        m2[i][j]) {
            return 0;
        }
    }
}
return 1;
}

```

```

void verifyAdditionAssociativity(void) { int m, n;
    printf("Enter dimensions m and n for matrices A, B, and C: "); scanf("%d %d", &m, &n);
    if (m <= 0 || n <= 0) return;

    int A[m][n], B[m][n], C[m][n]; int AB[m][n],
    BC[m][n];
    int LHS[m][n], RHS[m][n];

    printf("Enter elements of Matrix A (%dx%d):\n", m, n); readMatrix(m, n, A);
    printf("Enter elements of Matrix B (%dx%d):\n", m, n); readMatrix(m, n, B);
    printf("Enter elements of Matrix C (%dx%d):\n", m, n); readMatrix(m, n, C);

    addMatrices(m, n, A, B, AB); addMatrices(m, n, AB,
    C, LHS);

    addMatrices(m, n, B, C, BC); addMatrices(m, n, A,
    BC, RHS);

    printf("\n(A + B) + C:\n");
}

```

```

printMatrix(m, n, LHS);

printf("\nA + (B + C):\n"); printMatrix(m, n, RHS);

if(compareMatrices(m, n, LHS, RHS)) {
    printf("\nSince (A+B)+C == A+(B+C), matrix addition is associative.\n");
} else {
    printf("\nMatrices do not match. Check input values.\n");
}
}

void verifyMultiplicationAssociativity(void) { int m, n, p, q;
    printf("Enter dimensions m, n, p, q: "); scanf("%d %d %d %d",
    &m, &n, &p, &q);
    if (m <= 0 || n <= 0 || p <= 0 || q <= 0) return;

    int A[m][n], B[n][p], C[p][q]; int AB[m][p],
    BC[n][q];
    int LHS[m][q], RHS[m][q];

    printf("Enter elements of Matrix A (%dx%d):\n", m, n); readMatrix(m, n, A);
    printf("Enter elements of Matrix B (%dx%d):\n", n, p); readMatrix(n, p, B);
    printf("Enter elements of Matrix C (%dx%d):\n", p, q); readMatrix(p, q, C);

    multiplyMatrices(m, n, p, A, B, AB); multiplyMatrices(m, p, q,
    AB, C, LHS);

    multiplyMatrices(n, p, q, B, C, BC); multiplyMatrices(m, n, q,
    A, BC, RHS);

    printf("\n(A * B) * C:\n"); printMatrix(m, q, LHS);

    printf("\nA * (B * C):\n"); printMatrix(m, q, RHS);

    if(compareMatrices(m, q, LHS, RHS)) {
        printf("\nSince (A*B)*C == A*(B*C), matrix multiplication is associative.\n");
    } else {
        printf("\nMatrices do not match. Check input values.\n");
    }
}
}

```